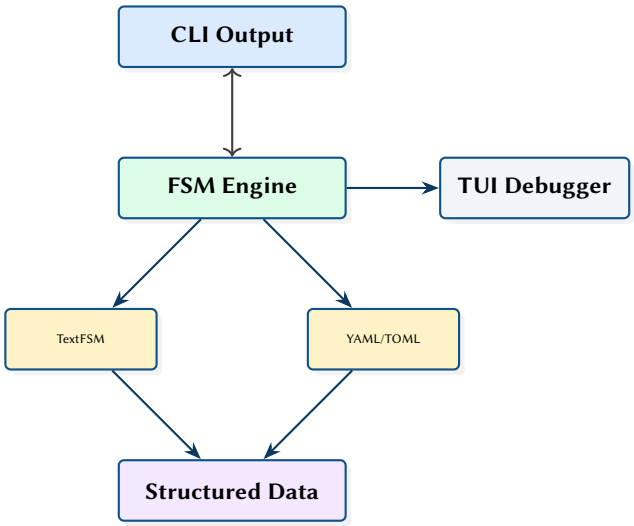
# cliscrape: A High-Performance CLI Parsing Engine with Interactive TUI Debugger

## TextFSM-Compatible State Machine Parser in Rust

Simon Knight

March 2026 • Version 0.1.1

Last updated: March 18, 2026




**Abstract.** Network automation pipelines rely heavily on parsing unstructured CLI output from network devices. This report presents cliscrape, a Rust implementation of a TextFSM-compatible state machine parser that provides over 4 million lines/sec throughput, offering a 10–50× performance improvement over the Python reference implementation. It maintains full compatibility with the existing NTC-Templates library, incorporates an embedded XDG-compliant template library, and features robust edge-case hardening. The system also introduces a modern YAML/TOML template format, a TUI debugger, and a radically innovative isomorphic generation mode that allows for bidirectional synthetic generation of CLI text from structured JSON state.

# Contents

# 1 Introduction and Ecosystem Context

The networking industry relies on CLI-based management for device configuration and monitoring. Parsing this unstructured text output is often the bottleneck in automation pipelines. TextFSM [1] provides a template-based approach using state machines, and the NTC-Templates [2] library offers hundreds of community-maintained templates.

However, the Python TextFSM implementation has three core limitations: performance limitations due to interpreted execution (often struggling with large `show tech-support` outputs), readability issues caused by dense inline regex syntax, and a lack of observability when templates fail to match expected output.

## 1.1 Ecosystem Role

`cliscrape` operates within the Operations Path of the broader network automation ecosystem. It sits directly behind the connectivity layer, transforming raw device output into structured operational state.

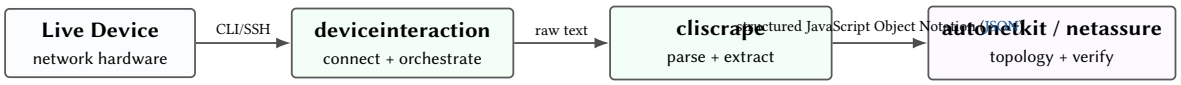



**Figure 1.** The Operations Path pipeline. `deviceinteraction` handles connectivity, while `cliscrape` handles pure parsing, feeding operational state to analysis tools.

# 2 Operational Model and Case Studies

The foundational goal of `cliscrape` is taking semi-structured CLI text and enforcing a typed schema contract. Consider a standard `show version` command from a Cisco IOS device:

Raw CLI Output (Cisco IOS)

```
Cisco IOS Software, C2960 Software (C2960-LANBASEK9-M), Version 15.0(2)SE11, RELEASE SOFTWARE (fc3
    ↪ )
ROM: Bootstrap program is C2960 boot loader
SW-CORE-01 uptime is 5 weeks, 2 days, 3 hours, 45 minutes
System image file is "flash:/c2960-lanbasek9-mz.150-2.SE11.bin"
```

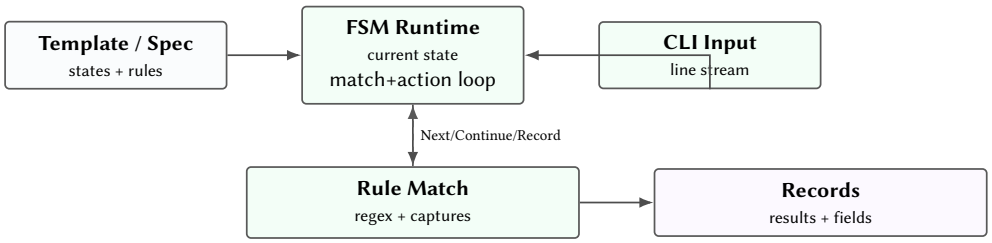
Instead of brittle custom scripts or string-splitting, the Finite State Machine (FSM) engine applies an embedded YAML or TextFSM template. The variables defined in the template dictate the keys of the resulting JSON object.

Parsed Structured Output

```
[
  {
    "hostname": "SW-CORE-01",
    "model": "WS-C2960-48TT-L",
    "serial": ":",
    "uptime": "5 weeks, 2 days, 3 hours, 45 minutes",
    "version": "15.0(2)SE11,"
  }
]
```

This operational model allows downstream tools like `netassure` to perform strict type-checking and logical assertions against properties like `uptime` and `version`, entirely decoupled from the vendor-specific formatting quirks of the original CLI.

# 3 FSM Engine Architecture




**Control vs data.** The same regex match both extracts data (captures) and drives control (state transitions). This is what makes TextFSM robust to prompts, pagination, and error banners.

**Figure 2.** FSM engine model: rule matching is both evidence extraction and control-flow.

The core engine implements the exact TextFSM state machine semantics using Rust.

**TextFSM semantics as a state machine over (line index, record state)**

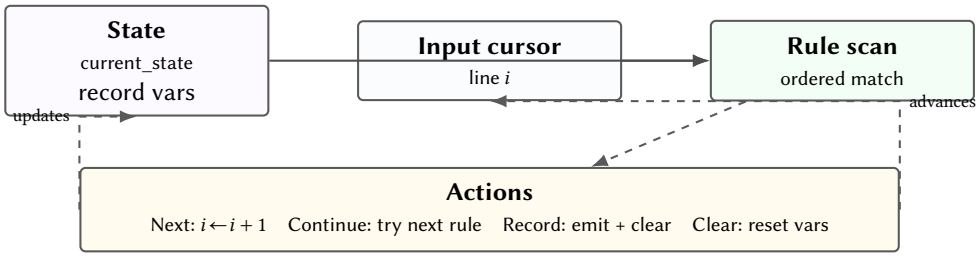



**Figure 3.** TextFSM parsing as an interpreter over a line cursor and an accumulating record.

For each input line, the engine evaluates rules in the current state sequentially. On match, captured groups are assigned to named variables, and the defined action (Next, Continue, Record, Clear) is executed.

### 3.1 Stateful Modifiers

Simple regular expressions are insufficient for network CLI outputs that exhibit hierarchical or stateful formatting (e.g., a routing table where a single routing protocol applies to multiple subsequent subnet lines). To support this, the engine implements TextFSM's stateful variable modifiers:

- **`Filldown`:** Retains a captured variable's value across multiple record emissions until explicitly cleared or overwritten.
- **`List`:** Rather than overwriting a variable on subsequent matches, accumulates the matched values into a native JSON array.

These modifiers allow the parser to build deep, relational data structures from flat text without requiring complex post-processing.

## 4 Template Formats

*Insight:* *The Rust `regex` crate's DFA-based engine provides predictable $O(n)$ matching per line, avoiding the catastrophic backtracking possible with Python's `re` module on pathological patterns.*

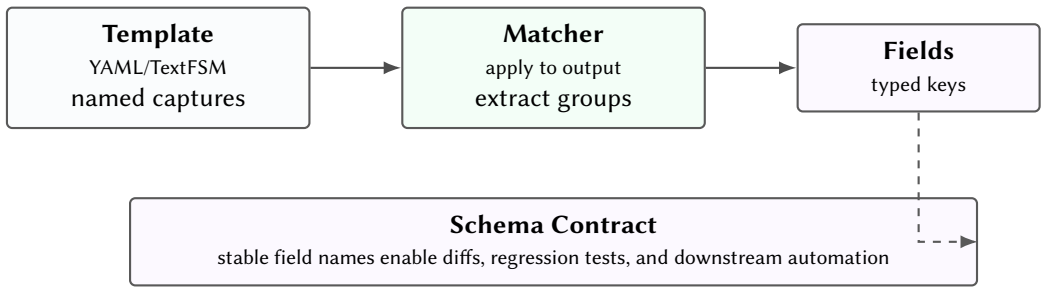



**Figure 4.** Templates define the parsing contract: semistructured CLI output becomes stable, typed records.

Supporting both TextFSM and YAML formats allows teams to migrate incrementally. Existing templates work immediately via a compatibility translation layer. New templates can use the more readable YAML format. Both compile to the same internal FSM representation (IR).

Beyond mere readability, the shift to YAML/TOML introduces a strict schema definition capability. Because values can be natively typed (e.g., explicitly defining an interface MTU as an integer rather than an arbitrary regex string), `cliscrape` can automatically generate downstream tooling assets like typed struct definitions, upstream documentation, and the embedded Template Catalog.

### 4.1 Native Type Coercion

Legacy Python TextFSM implementations fundamentally treat all captured data as strings. If a device outputs an MTU of `1500`, the resulting JSON output will be `"mtu": "1500"`. This pushes the burden of data normalisation onto downstream analysis tools.

Because `cliscrape`'s modern templates allow explicit type definitions, the engine performs native type coercion during the match phase. An extracted string representing a number is safely converted and emitted as a native JSON Integer. This seemingly small feature represents a major architectural win for the Operations Path: when data reaches `netassure`, it is instantly queryable without secondary casting operations.

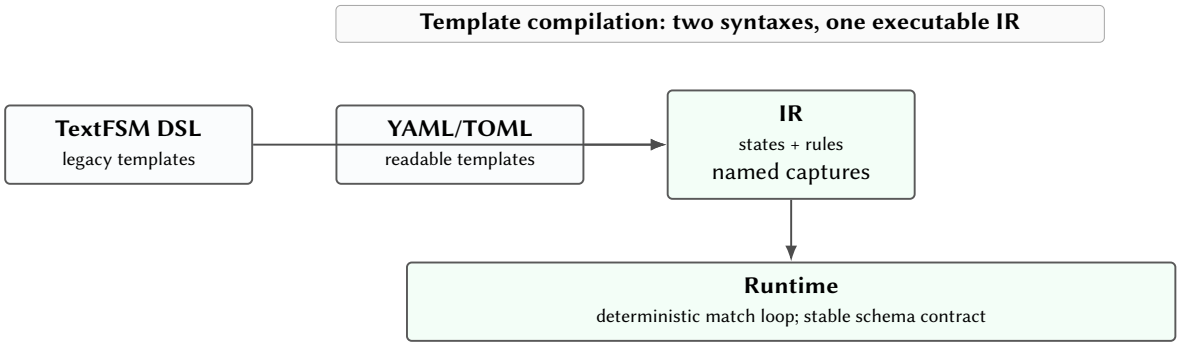



**Figure 5.** Compilation boundary: both template syntaxes compile into one IR consumed by the same runtime.

## 5 Production Hardening

Moving from a parsing prototype to a robust operational tool required several critical system enhancements to handle edge cases, simplify deployments, and ensure continuous availability.

### 5.1 Zero-Dependency Static Deployment

In enterprise networking environments, deploying tools to jump-hosts or restricted bastion servers is often hindered by Python runtime dependencies, virtual environments, and missing `pip` packages. Because `cliscrape` is written in Rust and bundles its templates internally, it compiles down to a single, zero-dependency static binary. Operators can simply `scp` the executable to any restricted node and parse CLI outputs out-of-the-box without altering the host system state.

### 5.2 Embedded Template Library & Hot-Reloading

To eliminate the friction of distributing raw template files, `cliscrape` ships with an embedded library of validated templates. Using an XDG-compliant resolver, users can parse outputs simply by declaring the target device and command, e.g., `-t cisco_ios_show_version`. User-defined overrides placed in `~/.local/share/cliscrape/templates/` automatically take precedence over embedded defaults.

Furthermore, the embedded library utilizes `rust-embed` to provide dual-mode execution. In release builds, templates are compiled directly into the binary for zero-dependency deployment. In debug builds, the engine dynamically reads from the local filesystem, enabling instant hot-reloading of templates during development without requiring recompilation.

### 5.3 Validation and Edge Case Management

Network CLI data is notoriously inconsistent due to firmware patches, unexpected banners, and hardware variants. The engine implements strict thresholds and boundaries:

- **Coverage Validation:** A configurable warning system triggers if a parsed output matches fewer fields than a minimum acceptable threshold (e.g., 80%), acting as a canary for firmware formatting changes.
- **Regex Boundary Enforcement:** To defend against catastrophic backtracking in custom user templates, timeout mechanisms and contextual line-number errors preemptively kill hung matching processes.
- **Snapshot Testing:** A comprehensive internal suite relies on `insta` snapshots, ensuring every embedded template guarantees identical parsed JSON parity against real-world fixture data across updates.

- **Execution Modes:** To balance resilience with strictness, the parser supports a Partial-Match mode (gracefully returning successfully parsed data alongside warnings for missing fields) and a `–strict` Fail-Fast mode that aborts execution upon the first error.

### 5.4 Structured Observability

For production deployments, raw text logs are insufficient. The engine utilises Rust's `tracing` framework to emit highly structured JSON events, capturing module-level lifecycle events, resolution paths, and parsing failures seamlessly for integration into enterprise observability pipelines.

## 6 Isomorphic Simulation and Synthetic Generation

The most significant architectural advancement in `cliscrape` is the introduction of isomorphic state machine execution. While traditional parsers represent a unidirectional projection from unstructured text to structured data, `cliscrape` treats the FSM as a bijective mapping ($FSM \leftrightarrow FSM^{-1}$).

### 6.1 The Inverse Parser ($FSM^{-1}$)

Because the modern YAML/TOML templates enforce strict schema contracts and type coercion, the parsing engine can be executed in reverse. In this mode, `cliscrape` consumes a structured JSON object and uses the template's rules to synthesize perfectly formatted, pixel-accurate raw CLI text.

This capability transforms the parser into a deterministic network device simulator. By providing a hypothetical device state (e.g., a BGP neighbor down or an interface flapping), operators can instantly generate the corresponding CLI output required to test downstream automation pipelines, monitoring logic, and alert systems without requiring access to physical hardware.

### 6.2 Mathematical Verification

Isomorphic generation also provides a mechanism for mathematical verification of templates. A template is considered "bijectively stable" if a raw CLI string parsed into JSON can be re-generated into a string that is semantically identical to the original input. This "round-trip" validation ensures that the template captures 100% of the relevant operational state without loss of fidelity.

## 7 Interactive TUI Debugger

The built-in Terminal User Interface (TUI) provides an integrated "Live Lab", offering visual, interactive observability into the parser execution, eliminating trial-and-error template development.

Beyond simply debugging a single file, the TUI includes a full **Template Browser**. Users can browse the embedded XDG library, select a template (e.g., `cisco_ios_show_version`), and instantly load it into the Live Lab alongside their current raw CLI text. The parsing engine automatically recompiles the selected template and re-evaluates the input text in real-time.

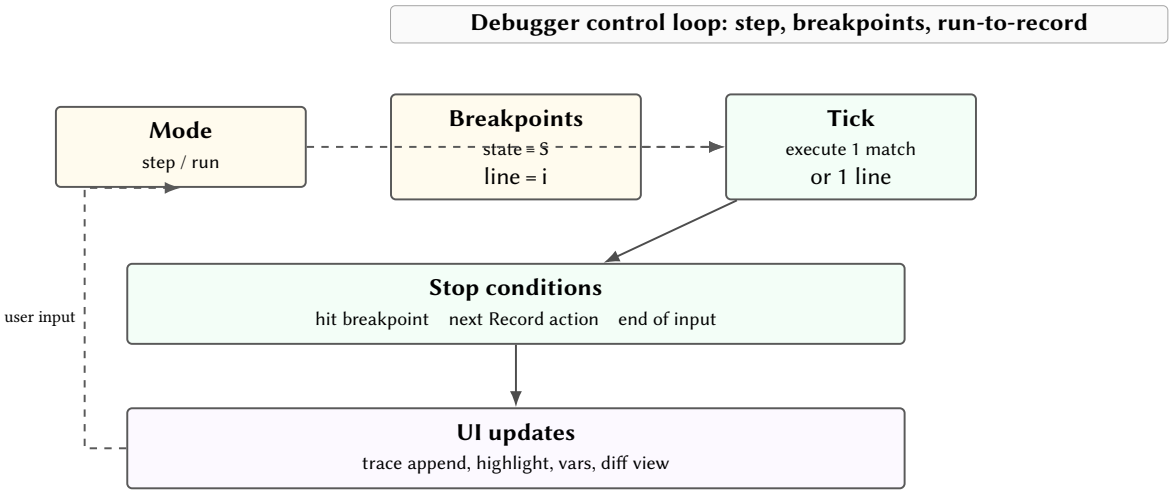



**Figure 6.** Debugger as a control loop over the parsing interpreter.

The interface offers four panes: an **Input Pane** for raw text with line highlighting, a **State Pane** detailing the FSM transition history, a **Variables Pane** showing live value states, and a **Diff View** explicitly highlighting matching capture groups.

## 8 Performance Evaluation

The transition from interpreted Python execution to a zero-cost abstraction model in Rust yields significant performance gains. Benchmark testing using the Criterion framework indicates that `cliscrape` operates at a sustained throughput of approximately **4.1 million lines per second**.

This magnitude of throughput enables the processing of massive `show tech-support` archives in milliseconds rather than seconds. When compared directly to the legacy Python implementation, the compiled Rust architecture and DFA-based regex matching deliver a stable **10–50× improvement** in parsing speed, free from GIL bottlenecks and catastrophic backtracking delays. Furthermore, the verified overhead of the production structured logging framework remains strictly below the targeted 5% margin.

## 9 Design Summary

**Table 1.** Key design decisions.

| Decision | Rationale |
|---|---|
| Rust regex crate | DFA-based $O(n)$ matching; no backtracking vulnerabilities |
| Full TextFSM compat | Zero-cost migration from existing NTC-Templates |
| Stateful modifiers | Filldown/List support native relational data extraction |
| Dual template formats | Incremental adoption; YAML for new templates |
| Native type coercion | YAML template hints yield directly queryable JSON types |
| Isomorphic Simulation | Bidirectional $FSM^{-1}$ execution enables synthetic CLI generation |
| Embedded library | XDG-compliant, zero-configuration parsing out-of-the-box |
| TUI debugger | Real-time FSM visualisation eliminates trial-and-error template development |

## References


[1] Google, *TextFSM: Template-based state machine for parsing*, 2024. [Online]. Available: https://github.com/google/textfsm

[2] Network to Code, *NTC Templates: Textfsm templates for network devices*, 2024. [Online]. Available: https://github.com/networktocode/ntc-templates

## Revision History

| Version | Date | Changes |
|---|---|---|
| 0.1.0 | March 2026 | Initial architecture report |
| 0.1.1 | March 2026 | Added ecosystem context, case studies, production hardening, and performance data |